

LAB GUIDE WITH SOLUTIONS

Redis Cluster Labs

Seven hands-on labs in Docker, with solutions inline
Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this lab guide

Seven labs, sized for a half-day hands-on session on Redis Cluster: building a six-node cluster from plain containers, reading how keys are spread across it, breaking it on purpose, growing and shrinking it, and finally running it from a compose file and connecting to it from Node.js. Every lab states its objective, shows the exact commands and the output you should see. Solutions are inline - this document is the instructor copy and the participant copy at the same time.

Labs C.1 to C.6 build **one** running cluster step by step - do them in order in a single session. Lab C.7 tears that cluster down and starts the ready-made one in `cluster_docker/`.

Lab	Title	Covers	Time
C.1	Six cluster-enabled containers	Deck 8 §2	20 min
C.2	Create the cluster and read it	Deck 8 §2	30 min
C.3	Slots, MOVED and hash tags	Deck 8 §1	35 min
C.4	Automatic and manual failover	Deck 8 §3	35 min
C.5	Losing a shard, reading from replicas	Deck 8 §3	25 min
C.6	Scale out, scale in - and fix a broken reshard	Deck 8 §4	40 min
C.7	Compose, and a Node.js client on the host	Deck 8 §2, §5	35 min

BEFORE YOU START Docker Desktop running, and a bash shell - Git Bash or WSL on Windows. The loops in these labs are bash; PowerShell users can run the six docker commands one at a time. Ports 7101-7108 must be free.

READ THIS Lines starting with \$ run in your own shell; lines starting with > run inside an interactive redis-cli. Output was captured non-interactively, so it shows errors without the (error) prefix and strings without quotes - an interactive redis-cli adds both. Node IDs and 172.18.0.x addresses are generated per run: yours will differ, the shape will not.

LAB C.1 · 20 MINUTES

Six Cluster-Enabled Containers

Objective	Prerequisite
Start six Redis servers in cluster mode and see what a cluster node looks like before it belongs to a cluster.	Docker Desktop running. No prior lab needed.

Step 1 · A network, then six nodes

A user-defined network lets the nodes find each other by container name. Each node publishes its client port on the host as 7101 to 7106.

```
$ docker network create icici-cluster-net
$ for i in 1 2 3 4 5 6; do
    docker run -d --name redis-node$i --hostname redis-node$i \
        --network icici-cluster-net -p 710$i:6379 \
        redis:7-alpine redis-server --cluster-enabled yes \
        --cluster-config-file nodes.conf --cluster-node-timeout 5000 --appendonly yes
done
$ docker ps --filter name=redis-node --format '{{.Names}} {{.Ports}}'
redis-node6 0.0.0.0:7106->6379/tcp
...
redis-node1 0.0.0.0:7101->6379/tcp
```

Step 2 · Ask a node about its cluster

```
$ docker exec redis-node1 redis-cli CLUSTER INFO
cluster_state:fail
cluster_slots_assigned:0
cluster_slots_ok:0
cluster_known_nodes:1
cluster_size:0
$ docker exec redis-node1 redis-cli SET icici:k v
CLUSTERDOWN Hash slot not served
$ docker exec redis-node1 redis-cli CLUSTER NODES
8262e3b9b5e18b97889aea960d636a0ecdf1f3a7 :6379@16379 myself,master - 0 0 0 connected
```

Step 3 · Read the startup log

```
$ docker logs redis-node1 2>&1 | grep 'No cluster'
No cluster configuration found, I'm 8262e3b9b5e18b97889aea960d636a0ecdf1f3a7
$ docker logs redis-node1 2>&1 | grep databases
WARNING: Changing databases number from 16 to 1 since we are in cluster mode
```

Solution · Questions

Why does SET fail? Every key belongs to one of 16384 slots, and this node owns none of them - nobody does yet. A cluster node refuses writes for a slot nobody serves rather than accept data that would be invisible once slots are assigned.

What is the 40-character string? The node ID, generated on first start and written to `/data/nodes.conf`. From here on the cluster identifies this node by that ID - not by its container name and not by its IP.

What is @16379? The cluster bus: client port + 10000. Nodes gossip, report failures and vote over it. It is not published to the host because no client ever uses it.

CHECKPOINT Six nodes running, each reporting cluster_state:fail, cluster_known_nodes:1 and zero slots.

LAB C.2 · 30 MINUTES

Create the Cluster and Read It

Objective	Prerequisite
Turn six strangers into a 3-master, 3-replica cluster with one command, then learn to read CLUSTER INFO, CLUSTER NODES and --cluster check.	Lab C.1 - six nodes running.

Step 1 · --cluster create

```
$ docker exec redis-node1 redis-cli --cluster create \
  redis-node1:6379 redis-node2:6379 redis-node3:6379 \
  redis-node4:6379 redis-node5:6379 redis-node6:6379 \
  --cluster-replicas 1 --cluster-yes
>>> Performing hash slots allocation on 6 nodes...
Master[0] -> Slots 0 - 5460
Master[1] -> Slots 5461 - 10922
Master[2] -> Slots 10923 - 16383
Adding replica redis-node5:6379 to redis-node1:6379
Adding replica redis-node6:6379 to redis-node2:6379
Adding replica redis-node4:6379 to redis-node3:6379
M: 8262e3b9b5e18b97889aea960d636a0ecdf1f3a7 redis-node1:6379
  slots:[0-5460] (5461 slots) master
...
>>> Nodes configuration updated
[OK] All 16384 slots covered.
```

WATCH OUT --cluster-yes answers redis-cli's confirmation prompt for you. Leave it off the first time you do this on real servers and read the proposed plan before accepting it.

Step 2 · Is it healthy?

```
$ docker exec redis-node1 redis-cli CLUSTER INFO | head -8
cluster_state:ok
cluster_slots_assigned:16384
cluster_slots_ok:16384
cluster_slots_pfail:0
cluster_slots_fail:0
cluster_known_nodes:6
cluster_size:3
cluster_current_epoch:6
```

Step 3 · Who owns what

```
$ docker exec redis-node1 redis-cli CLUSTER NODES | sort -k3
ee798def... 172.18.0.4:6379@16379 master - 0 1790214645000 3 connected 10923-16383
b7e17bae... 172.18.0.3:6379@16379 master - 0 1790214645548 2 connected 5461-10922
8262e3b9... 172.18.0.2:6379@16379 myself,master - 0 0 1 connected 0-5460
7bd7d3dc... 172.18.0.6:6379@16379 slave 8262e3b9... 0 1790214644942 1 connected
4f1fea46... 172.18.0.7:6379@16379 slave b7e17bae... 0 1790214645952 2 connected
81b30345... 172.18.0.5:6379@16379 slave ee798def... 0 1790214644000 3 connected
```

```
$ docker exec redis-node1 redis-cli --cluster check redis-node1:6379
redis-node1:6379 (8262e3b9...) -> 0 keys | 5461 slots | 1 slaves.
172.18.0.4:6379 (ee798def...) -> 0 keys | 5461 slots | 1 slaves.
172.18.0.3:6379 (b7e17bae...) -> 0 keys | 5462 slots | 1 slaves.
[OK] 0 keys in 3 masters.
...
[OK] All nodes agree about slots configuration.
>>> Check for open slots...
>>> Check slots coverage...
[OK] All 16384 slots covered.
```

Exercise · Draw the cluster

From the CLUSTER NODES output alone, draw the three master/replica pairs with each master's slot range. Then look up each IP with `docker inspect -f '{{.Name}} {{(index .NetworkSettings.Networks "icici-cluster-net").IPAddress}}' redis-node1` and label the boxes with container names.

Solution

Master	Slots	Its replica
redis-node1 (172.18.0.2)	0 - 5460	redis-node5 (172.18.0.6)
redis-node2 (172.18.0.3)	5461 - 10922	redis-node6 (172.18.0.7)
redis-node3 (172.18.0.4)	10923 - 16383	redis-node4 (172.18.0.5)

Read a replica line right to left: `slave 8262e3b9...` names the master it copies. Note node4 is **not** node1's replica even though it was listed fourth - `redis-cli` chose the pairing, which is why you read the output rather than assume it.

CHECKPOINT cluster_state:ok, 16384 slots covered, and a drawing of which replica backs which master.

LAB C.3 · 35 MINUTES

Slots, MOVED and Hash Tags

Objective	Prerequisite
See a node refuse a key it does not own, let redis-cli follow the redirect, and use hash tags to make a multi-key command legal.	Lab C.2 - a healthy cluster.

Step 1 · Where does a key live?

```
$ X="docker exec redis-node1 redis-cli"
$ $X CLUSTER KEYSLOT icici:acct:1001
12170
$ $X CLUSTER KEYSLOT icici:acct:1002
8169
$ $X CLUSTER KEYSLOT icici:bal:9003
6348
```

Step 2 · Without -c, then with it

```
$ $X SET icici:acct:1001 Asha
MOVED 12170 172.18.0.4:6379
$ $X -c SET icici:acct:1001 Asha
OK
$ printf 'SET icici:acct:1002 Ravi\nGET icici:acct:1002\nGET icici:acct:1001\n' \
  | docker exec -i redis-node1 redis-cli -c
-> Redirected to slot [8169] located at 172.18.0.3:6379
OK
Ravi
-> Redirected to slot [12170] located at 172.18.0.4:6379
Asha
```

Three commands, one connection: after the first redirect the client stays on node2, so the second GET needs no redirect - and the third moves on to node3.

Step 3 · Multi-key commands

```
$ $X -c MSET icici:acct:1001 a icici:acct:1002 b
CROSSSLOT Keys in request don't hash to the same slot
$ $X CLUSTER KEYSLOT "{icici:cust:42}:profile"
2022
$ $X CLUSTER KEYSLOT "{icici:cust:42}:balance"
2022
$ $X -c MSET "{icici:cust:42}:profile" Asha "{icici:cust:42}:balance" 5000 \
  "{icici:cust:42}:txns" 3
OK
$ $X -c MGET "{icici:cust:42}:profile" "{icici:cust:42}:balance"
Asha
5000
```

WATCH OUT Quote hash-tag keys in the shell - bash treats {a,b} as brace expansion.

Step 4 · Load 3000 keys and look at the spread

```
$ docker exec redis-node1 sh -c \
  'for i in $(seq 1 3000); do echo "SET icici:txn:$i $i"; done | redis-cli -c > /dev/null'
$ for i in 1 2 3; do echo "node$i $(docker exec redis-node$i redis-cli DBSIZE)"; done
node1 1011
node2 1002
node3 992
$ docker exec redis-node1 redis-cli --cluster info redis-node1:6379
[OK] 3005 keys in 3 masters.
0.18 keys per slot on average.
```

Step 5 · Habits that break

```
$ $X SELECT 1
ERR SELECT is not allowed in cluster mode
$ for i in 1 2 3; do echo "node$i: $(docker exec redis-node$i redis-cli KEYS 'icici:acct:*')"; done
node1:
node2: icici:acct:1002
node3: icici:acct:1001
```

Solution · Questions

Why 3005 keys and not 3000? The three {icici:cust:42} keys plus icici:acct:1001 and icici:acct:1002 - five more.

Why is node1's DBSIZE the largest? Slot 2022 is on node1, so the three hash-tagged customer keys all landed there. That is also the risk of hash tags: one enormous tag, such as {icici} on every key, would put the whole dataset in one slot on one master.

Name a key design for a customer's profile, balance and last 10 transactions that lets a MULTI update all three. Put the customer ID in the tag: {cust:42}:profile, {cust:42}:balance, {cust:42}:txns. Different customers still spread across the cluster.

CHECKPOINT You can predict whether a multi-key command will get CROSSSLOT before you run it.

LAB C.4 · 35 MINUTES

Automatic and Manual Failover

Objective	Prerequisite
Kill a master, watch the cluster promote its replica with no help, bring the old master back, and then hand its role back gracefully.	Lab C.3 - a healthy cluster holding ~3005 keys.

Step 1 · Stop a master

Open a second terminal and follow node5's log, then stop node1 in the first:

```
$ docker logs -f redis-node5      # terminal 2
$ docker stop redis-node1         # terminal 1
```

```
# node5's log, measured - its link to node1 dropped at 01:51:55.657
01:51:55.657 Reconnecting to MASTER 172.18.0.2:6379
01:52:02.532 FAIL message received from ee798def... about 8262e3b9...
01:52:02.532 Cluster state changed: fail
01:52:02.607 Start of election delayed for 808 milliseconds (rank #0, offset 43859).
01:52:03.515 Starting a failover election for epoch 7.
01:52:03.660 Failover election won: I'm the new master.
```

Step 2 · Check from a surviving node

```
$ docker exec redis-node2 redis-cli CLUSTER NODES | sort -k2
8262e3b9... 172.18.0.2:6379@16379 master,fail - ... 1 connected
7bd7d3dc... 172.18.0.6:6379@16379 master - 0 1790214797515 7 connected 0-5460
...
$ docker exec redis-node2 redis-cli CLUSTER INFO | grep -E 'state|epoch'
cluster_state:ok
cluster_current_epoch:7
$ docker exec redis-node5 redis-cli DBSIZE
1011
```

Every key node1 held is still there - on node5, which already had them all as a replica.

Step 3 · The old master comes back

```
$ docker start redis-node1
$ docker exec redis-node1 redis-cli INFO replication | grep -E '^role|master_host|link'
role:slave
master_host:172.18.0.6
master_link_status:up
$ docker logs redis-node1 | grep -E 'Configuration change|resync'
Configuration change detected. Reconfiguring myself as a replica of 7bd7d3dc...
Full resync from master: ab09857521d3b3cbe69442cef2d2630db71568da:43949
```

Step 4 · Give node1 its role back - gracefully

```
$ docker exec redis-node1 redis-cli CLUSTER FAILOVER
OK
$ sleep 5
$ for i in 1 5; do echo "node$i $(docker exec redis-node$i redis-cli INFO replication | grep ^role)"; done
node1 role:master
node5 role:slave
$ docker exec redis-node2 redis-cli CLUSTER FAILOVER
ERR You should send CLUSTER FAILOVER to a replica
```

MEASURED TRAP CLUSTER FAILOVER returns OK as soon as it *starts*. In one of our runs the election took 2.5 s after the OK. Always wait and re-check the roles before your next step - sending a second CLUSTER FAILOVER to the node that has meanwhile become the replica again swaps them straight back.

Solution · Questions

How long was the cluster unable to serve slots 0-5460? From node1 going down (01:51:55.66, when node5 lost its link) to the election win (01:52:03.66): about 8.0 s. About 6.9 s of that is waiting to be sure - cluster-node-timeout is 5000 ms, and the masters must agree (quorum) before anyone is marked FAIL. The election itself took about 1 s, most of it the deliberate 808 ms delay.

Why did node1 need a full resync, not a partial one? node5 became a master with a new replication history the moment it won the election. node1's history no longer matches it, so - exactly as in deck 7 - Redis refuses to guess and copies everything.

Automatic failover vs CLUSTER FAILOVER? Automatic is for a dead master and can lose the last writes the replica had not yet received. CLUSTER FAILOVER is for maintenance: the replica first catches up to the master's exact offset, so nothing is lost.

CHECKPOINT You have timed a failover from the logs and can explain every line of it.

LAB C.5 · 25 MINUTES

Losing a Shard, Reading From Replicas

Objective	Prerequisite
See what the cluster does when a master AND its replica are both gone, and let a replica serve reads.	Lab C.4 - node1 back as master, cluster ok.

Step 1 · Stop a whole shard

```
$ docker exec redis-node6 redis-cli INFO replication | grep master_host
master_host:172.18.0.3           # node6 is node2's replica
$ docker stop redis-node2 redis-node6
$ sleep 9
$ docker exec redis-node1 redis-cli CLUSTER INFO | grep -E 'state|slots_ok|slots_fail'
cluster_state:fail
cluster_slots_ok:10923
cluster_slots_fail:5461
$ docker exec redis-node1 redis-cli -c SET icici:acct:1002 x           # slot 8169, lost
CLUSTERDOWN The cluster is down
$ docker exec redis-node1 redis-cli -c GET "{icici:cust:42}:profile"   # slot 2022, healthy
CLUSTERDOWN The cluster is down
```

Step 2 · Accept partial service

```
$ for i in 1 3 4 5; do docker exec redis-node$i redis-cli \
  CONFIG SET cluster-require-full-coverage no; done
$ docker exec redis-node1 redis-cli CLUSTER INFO | grep state
cluster_state:ok
$ docker exec redis-node1 redis-cli -c GET "{icici:cust:42}:profile"
Asha
$ docker exec redis-node1 redis-cli -c GET icici:acct:1002
Could not connect to Redis at 172.18.0.3:6379: Host is unreachable
```

Step 3 · Restore

```
$ for i in 1 3 4 5; do docker exec redis-node$i redis-cli \
  CONFIG SET cluster-require-full-coverage yes; done
$ docker start redis-node2 redis-node6 ; sleep 8
$ docker exec redis-node1 redis-cli CLUSTER INFO | grep -E 'state|slots_ok'
cluster_state:ok
cluster_slots_ok:16384
$ docker exec redis-node1 redis-cli -c GET icici:acct:1002
Ravi
```

Step 4 · Reads from a replica

node5 replicates node1, which owns slot 2022. Pipe the commands into one redis-cli so they share a connection - READONLY is per connection.

```
$ printf 'GET {icici:cust:42}:profile\nREADONLY\nGET {icici:cust:42}:profile\n
  SET {icici:cust:42}:profile x\nREADWRITE\nGET {icici:cust:42}:profile\n' \
  | docker exec -i redis-node5 redis-cli
MOVED 2022 172.18.0.2:6379
```

```
OK
Asha
MOVED 2022 172.18.0.2:6379
OK
MOVED 2022 172.18.0.2:6379
```

Solution · Questions

Why did the healthy slot 2022 also refuse service? With cluster-require-full-coverage yes (the default) a cluster that cannot serve every slot stops serving any. The alternative is an application that gets "not found" for a third of its keys and cannot tell that from a genuinely missing key.

When is 'no' reasonable? A cache, where a miss just means going to the database. Not for balances.

Line by line, Step 4: GET redirected (a replica does not serve reads by default) · READONLY accepted · GET served locally · SET redirected (writes always go to the master) · READWRITE accepted · GET redirected again.

Why was nothing lost? The containers were stopped, not removed. Each node's AOF and nodes.conf survived in its container filesystem - Lab C.7 moves them onto volumes.

CHECKPOINT You can explain why the default is to stop, and when a replica may answer a read.

LAB C.6 · 40 MINUTES

Scale Out, Scale In - and Fix a Broken Reshard

Objective	Prerequisite
Add a master and its replica, move slots to it, balance the cluster, then empty the new master and remove both nodes - recovering from a reshard that fails halfway.	Lab C.5 - a healthy cluster holding 3005 keys.

Step 1 · Two more nodes, then add node7 as an empty master

```
$ for i in 7 8; do docker run -d --name redis-node$i --hostname redis-node$i \
  --network icici-cluster-net -p 710$i:6379 redis:7-alpine redis-server \
  --cluster-enabled yes --cluster-config-file nodes.conf \
  --cluster-node-timeout 5000 --appendonly yes; done
$ X="docker exec redis-node1 redis-cli"
$ $X --cluster add-node redis-node7:6379 redis-node1:6379
>>> Send CLUSTER MEET to node redis-node7:6379 to make it join the cluster.
[OK] New node added correctly.
$ $X --cluster info redis-node1:6379 | grep 0.8
172.18.0.8:6379 (d61167d8...) -> 0 keys | 0 slots | 0 slaves.
$ N7=$(docker exec redis-node7 redis-cli CLUSTER MYID | tr -d '\r')
```

Step 2 · Reshard 1000 slots to it - what happened to us

We ran the reshard about three seconds after add-node:

```
$ $X --cluster reshard redis-node1:6379 --cluster-from all \
  --cluster-to $N7 --cluster-slots 1000 --cluster-yes
...
Moving slot 11254 from ee798def03ab5c0fc07c582aaeb10d4170f0d823
Moving slot 11255 from ee798def03ab5c0fc07c582aaeb10d4170f0d823
Node 172.18.0.3:6379 replied with error:
ERR Target instance replied with error: CLUSTERDOWN The cluster is down
$ $X --cluster check redis-node1:6379 | grep -E 'WARN|OK'
[OK] All nodes agree about slots configuration.
[WARNING] Node 172.18.0.8:6379 has slots in importing state 5466.
[WARNING] Node 172.18.0.3:6379 has slots in migrating state 5466.
[WARNING] The following slots are open: 5466.
```

Every node except the newcomer already said cluster_state:ok. node7 had joined but had not yet learned the whole topology, so it refused the incoming keys - and the move stopped with slot 5466 half-migrated: *migrating* on node2, *importing* on node7. If yours succeeded first time, skip to Step 4.

Step 3 · Repair, then wait for the newcomer

```
$ $X --cluster fix redis-node1:6379 --cluster-yes
>>> Fixing open slot 5466
Set as migrating in: 172.18.0.3:6379
Set as importing in: 172.18.0.8:6379
>>> Case 1: Moving slot 5466 from 172.18.0.3:6379 to 172.18.0.8:6379
[OK] All 16384 slots covered.
$ docker exec redis-node7 redis-cli CLUSTER INFO | grep cluster_state
cluster_state:ok
```

Step 4 · Reshard (again) and check

```
$ $X --cluster reshard redis-node1:6379 --cluster-from all \
  --cluster-to $N7 --cluster-slots 995 --cluster-yes > /dev/null
$ $X --cluster info redis-node1:6379
redis-node1:6379 (8262e3b9...) -> 950 keys | 5129 slots | 1 slaves.
172.18.0.8:6379 (d61167d8...) -> 179 keys | 1000 slots | 0 slaves.
172.18.0.3:6379 (b7e17bae...) -> 945 keys | 5125 slots | 1 slaves.
172.18.0.4:6379 (ee798def...) -> 931 keys | 5130 slots | 1 slaves.
[OK] 3005 keys in 4 masters.
```

995 rather than 1000 because node7 already held 5 slots from the interrupted run. It took 10 seconds; the key total did not change.

Step 5 · Give node7 a replica, then balance

```
$ $X --cluster add-node redis-node8:6379 redis-node1:6379 \
  --cluster-slave --cluster-master-id $N7
>>> Configure node as replica of 172.18.0.8:6379.
[OK] New node added correctly.
$ $X --cluster rebalance redis-node1:6379
>>> Rebalancing across 4 nodes. Total weight = 4.00
Moving 1034 slots from 172.18.0.4:6379 to 172.18.0.8:6379
Moving 1033 slots from redis-node1:6379 to 172.18.0.8:6379
Moving 1029 slots from 172.18.0.3:6379 to 172.18.0.8:6379
$ $X --cluster info redis-node1:6379 | head -4
redis-node1:6379 (8262e3b9...) -> 755 keys | 4096 slots | 1 slaves.
172.18.0.8:6379 (d61167d8...) -> 746 keys | 4096 slots | 1 slaves.
172.18.0.3:6379 (b7e17bae...) -> 754 keys | 4096 slots | 1 slaves.
172.18.0.4:6379 (ee798def...) -> 750 keys | 4096 slots | 1 slaves.
```

Step 6 · Scale back in

```
$ N8=$(docker exec redis-node8 redis-cli CLUSTER MYID | tr -d '\r')
$ $X --cluster rebalance redis-node1:6379 --cluster-weight $N7=0
>>> Rebalancing across 4 nodes. Total weight = 3.00
Moving 1366 slots from 172.18.0.8:6379 to redis-node1:6379
Moving 1365 slots from 172.18.0.8:6379 to 172.18.0.3:6379
Moving 1365 slots from 172.18.0.8:6379 to 172.18.0.4:6379
$ $X --cluster info redis-node1:6379
redis-node1:6379 (8262e3b9...) -> 1011 keys | 5462 slots | 1 slaves.
172.18.0.3:6379 (b7e17bae...) -> 1002 keys | 5461 slots | 1 slaves.
172.18.0.4:6379 (ee798def...) -> 992 keys | 5461 slots | 3 slaves.
$ $X --cluster del-node redis-node1:6379 $N8
>>> Sending CLUSTER FORGET messages to the cluster...
>>> Sending CLUSTER RESET SOFT to the deleted node.
$ $X --cluster del-node redis-node1:6379 $N7
$ docker rm -f redis-node7 redis-node8
$ $X CLUSTER INFO | grep -E 'state|known|size'
cluster_state:ok
cluster_known_nodes:6
cluster_size:3
```

Solution · Questions

Why does node3 briefly show 3 slaves? Once node7 owned no slots it was no longer a master worth keeping; it and its replica node8 were re-parented under node3 (Redis replica migration). del-node then removed them from the cluster.

Why are the key counts exactly the same as before Step 1 (1011 / 1002 / 992)? Keys travel with their slots, and the weight-0 rebalance handed slots back in the same ranges. The data never depended on which node held it.

What would you check before resharding in production? That every node - including the one you just added - reports cluster_state:ok, and that --cluster check shows no open slots. And afterwards, run --cluster check again: a reshard can stop halfway, as ours did.

CHECKPOINT You have grown the cluster to 4 masters, shrunk it back to 3, and repaired an interrupted slot move with --cluster fix.

Cleaning up before Lab C.7

```
$ docker rm -f redis-node1 redis-node2 redis-node3 redis-node4 redis-node5 redis-node6
$ docker network rm icici-cluster-net
```

WATCH OUT Lab C.7's compose file uses the same network name and host ports 7101-7106 - skip this clean-up and it will not start.

LAB C.7 · 35 MINUTES

Compose, and a Node.js Client on the Host

Objective	Prerequisite
Run the ready-made cluster from <code>cluster_docker/</code> , prove it survives down/up, and connect to it from Node.js on the Windows host - first the way that hangs, then the way that works.	Lab C.6 clean-up done. Node.js 18+ on the host.

Step 1 · Up

```
$ cd cluster_docker
$ docker compose up -d
$ docker compose ps --format '{{.Name}} {{.Status}}'
  icici-rc-node1 Up 8 seconds (healthy)
  ...
  icici-rc-node6 Up 9 seconds (healthy)
$ docker logs icici-rc-init | tail -1
[OK] All 16384 slots covered.
$ docker exec icici-rc-node1 redis-cli CLUSTER INFO | grep -E 'state|size|known'
cluster_state:ok
cluster_known_nodes:6
cluster_size:3
```

Read `docker-compose.yml` alongside this lab. Three things differ from Labs C.1-C.6: each node has a **fixed IP** (172.30.0.11-16), a **named volume** for `/data`, and a one-shot `cluster-init` service that runs `--cluster create` only if the cluster is not already formed.

Step 2 · Prove it survives down / up

```
$ docker exec icici-rc-node1 sh -c \
  'for i in $(seq 1 3000); do echo "SET icici:txn:$i $i"; done | redis-cli -c > /dev/null'
$ docker exec icici-rc-node1 redis-cli -c SET icici:acct:1001 Asha
$ ID1=$(docker exec icici-rc-node1 redis-cli CLUSTER MYID)
$ docker compose down          # removes every container and the network
$ docker compose up -d
$ docker logs icici-rc-init | tail -1
cluster already formed - nothing to do
$ docker exec icici-rc-node1 redis-cli CLUSTER INFO | grep state
cluster_state:ok
$ [ "$ID1" = "$(docker exec icici-rc-node1 redis-cli CLUSTER MYID)" ] && echo same id
same id
$ docker exec icici-rc-node1 redis-cli --cluster info 172.30.0.11:6379 | tail -2
[OK] 3001 keys in 3 masters.
0.18 keys per slot on average.
```

Step 3 · The Node.js client that hangs

```
$ cd client
$ npm install
$ npm run naive
no reply after 10s - the client is stuck dialling 172.30.0.x addresses
```

The client reached localhost:7101, asked for the slot map, and was told the masters live at 172.30.0.11, .12 and .13 - addresses inside Docker's network that the Windows host cannot reach. redis-cli never hit this because every command so far ran *inside* a container.

Step 4 · The client that works

```
$ npm start
connected - 3 masters
icici:acct:1001 -> Asha
icici:acct:1002 -> Ravi
MULTI on one slot -> Meera now has 1750
MULTI across slots -> MaxCommandRedirectionsError: Too many Cluster redirections
```

Open `cluster_client.js`. Two differences from `naive.js`: `nodeAddressMap`, which rewrites each advertised `172.30.0.x:6379` to `localhost:710x` - a map that can be written by hand only because compose pins the IPs - and `rootNodes` listing all six nodes rather than one. The exercise shows why the second one matters.

Exercise · Start the client while a master is down

Run `docker compose stop redis-node1`, wait 10 seconds, and run `npm start`. Then edit `rootNodes` down to just `redis://localhost:7101` and run it again. Finally start `node1` and check its role.

Solution

```
$ docker compose stop redis-node1 ; sleep 10
$ docker exec icici-rc-node2 redis-cli CLUSTER NODES | awk '{print $2, $3, $9}' | sort
172.30.0.11:6379@16379 master, fail
172.30.0.12:6379@16379 myself, master 5461-10922
172.30.0.13:6379@16379 master 10923-16383
172.30.0.14:6379@16379 slave
172.30.0.15:6379@16379 master 0-5460
172.30.0.16:6379@16379 slave
$ docker exec icici-rc-node2 redis-cli -c GET icici:acct:1001
Asha
$ docker compose start redis-node1 ; sleep 6
$ docker exec icici-rc-node1 redis-cli INFO replication | grep -E '^role|master_host'
role: slave
master_host: 172.30.0.15
```

```
$ npm start                                     # node1 down, six rootNodes
connected - 3 masters
icici:acct:1001 -> Asha
icici:acct:1002 -> Ravi
MULTI on one slot -> Meera now has 1750
$ npm start                                     # node1 down, rootNodes = [7101] only
RootNodesUnavailableError                       (thrown from cluster.connect())
```

With six seeds the client asks any live node for the slot map, learns that 172.30.0.15 (node5) now owns slots 0-5460, and 172.30.0.15 is already in its address map - nothing to change. With one seed, the one node it knows is the dead one, and the application cannot start at all even though the cluster is serving every slot. Always seed a cluster client with several nodes.

Cleaning up

```
$ docker compose down -v
$ docker volume ls -q | grep icici_rc || echo "no icici_rc volumes left"
no icici_rc volumes left
```

CHECKPOINT A cluster that survives down/up with the same node IDs, and a host-side Node.js client that routes keys, runs a single-slot transaction, and rides through a failover.

What you should be able to do now

- Start cluster-enabled Redis containers and form a 3-master, 3-replica cluster with one redis-cli command.
- Read CLUSTER INFO, CLUSTER NODES and --cluster check, and draw the topology from them.
- Predict a key's slot, explain MOVED, and design hash-tagged key names so multi-key commands and transactions stay legal.
- Time an automatic failover from the node logs, and perform a lossless CLUSTER FAILOVER.
- Explain cluster-require-full-coverage and when a replica may serve a read.
- Add and remove masters with add-node, reshard, rebalance and del-node - and repair an interrupted reshard with --cluster fix.
- Run a cluster under docker-compose that survives restarts, and connect to it from Node.js on the host with nodeAddressMap.

NEXT Every transcript in this guide came from real containers on the training machine. If a number on your machine differs - a node ID, an IP, a key count by a few - your machine is right; the shape of the output is what the lab is teaching.